

Agentic TI — Action Plan
TEAM 9-AIE B

ACTION PLAN

Agentic Threat Intelligence:

Autonomous Defense via Temporal Knowledge Graphs & Graph Reinforcement Learning

TEAM 9-AIE B

Department of Computer Science and Artificial Intelligence

June 2026

This document presents the system methodology for Agentic Threat Intelligence. It addresses key engineering risks, provides targeted solutions for each bottleneck, and introduces two cutting-edge architectural extensions — Interactive Policy Interrogation and Multi-Agent Adversarial Self-Play — designed to push this work to top-tier conference quality. Building on this baseline, the present revision incorporates the project team's consolidated feasibility evaluation, conducted independently for every phase and sub-step, together with the resulting design refinements adopted into the methodology.

Phases: 4 Phases + 2 Innovation Extensions

Novelty Axes: 5

Conference Target: IEEE S&P / ACM CCS (Full Paper)

1 Architecture Assessment Summary

This section documents the three categories of engineering and theoretical considerations that inform every subsequent design decision in this plan.

1.1 Category A — Engineering Risks

Risk 1 — Cascading Error Propagation. The pipeline is strictly sequential (TKG → TGN → DDQN → LLM). An NLP normalisation error in Phase 1 propagates into skewed embeddings in Phase 2, which forces the DDQN to make faulty decisions in Phase 3. A single point of failure at Layer 1 can invalidate the entire downstream pipeline.

Risk 2 — Neo4j Write Contention Under Streaming Load. High-frequency, real-time write loops from Apache Spark into Neo4j's transactional layer cause lock contention and transaction overhead. This risk is most acute at Week 3 and threatens the <5s end-to-end latency target set in Phase 4.

Risk 3 — DDQN Convergence Instability. In a dynamic graph-state environment, sparse rewards combined with the high dimensionality of TGN latent vectors frequently cause Double DQNs to plateau or diverge. Hyperparameter sweeps alone are insufficient mitigation for a two-week training window.

1.2 Category B — Theoretical Considerations

Gap 1 — Inductive Generalisation Boundary. A standard inductive test (masking “Ransomware” nodes) tests label novelty, not structural novelty. True Zero-Day exploits may introduce entirely new structural relationship patterns. If the structural fingerprint diverges sufficiently from training data, neighbourhood-similarity will fail silently without triggering any error signal.

Gap 2 — Reward Function Scale Mismatch. The reward combines Attacker Work Factor (integer graph path lengths), Operational Disruption (continuous traffic percentage), and False Positive indicators (binary). Without normalisation, one term will dominate gradient updates, rendering the Pareto frontier meaningless.

1.3 Category C — Innovation Extensions

Two extensions are proposed to elevate this system to a genuinely novel research contribution:

- Interactive Policy Interrogation — transforming the LLM layer from a static report generator into a counterfactual reasoning copilot for SOC analysts.
- Multi-Agent Adversarial Self-Play — training an adversary RL agent simultaneously with the defense agent to generate dynamic, generative Zero-Day scenarios without reliance on static log replays.

1.4 Category D — Feasibility Evaluation Methodology

Each phase and sub-step in Sections 3–6 below was independently evaluated against three criteria: feasibility (can it be implemented within the resourcing and timeline available), correctness (does the design solve the problem it claims to solve, for this project specifically), and novelty (does it constitute a genuine contribution beyond existing precedent). Where a step was found to be feasible but only incrementally novel, a refinement is proposed in place; where a step carried material implementation risk, an alternative formulation is proposed and assessed on the same three criteria. The consolidated findings of this evaluation are presented in Section 7.

2 System Architecture

2.1 Pipeline Diagram

The architecture employs two cross-cutting design choices visible at every layer: (a) an uncertainty channel carrying NLP confidence scores from Layer 1 through to Layer 3, and (b) an action-masking layer sitting between the TGN and the DDQN.

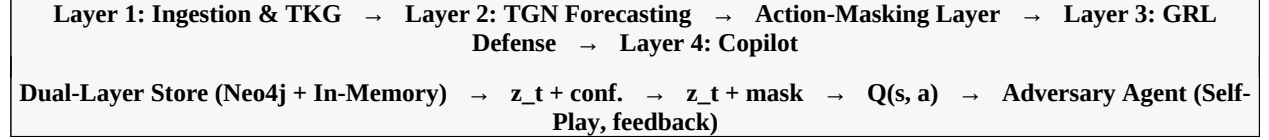


Figure 1: End-to-end pipeline with uncertainty channel and action-masking layer.

2.2 Component Design Decisions

Component	Design	Rationale
Data Store	Dual-layer: Neo4j (persistent) + In-Memory graph (streaming)	Decouples high-frequency writes from persistent querying
Triplet Output	Probabilistic, with extraction-confidence edge weight	Encodes NLP uncertainty explicitly
TGN Message Agg.	Confidence-weighted aggregation	Suppresses uncertain edges during embedding computation
RL State	Formally defined as $\langle z_t, A_{\text{valid}}(s_t) \rangle$	Incorporates topology constraints
Reward Function	Three unitless, normalised ratio terms	Produces a meaningful Pareto frontier
Layer 4	Interactive counterfactual copilot	Enables bidirectional analyst engagement
Training Environment	Static replay + live adversary agent	Dynamic, generative threat generation

Table 1: Component design decisions.

3 Phase 1 — Data Ingestion & TKG (Weeks 1–4)

Dual-data-layer architecture with uncertainty quantification on triplet extraction.

3.1 1A — Dual-Data-Layer Architecture

Streaming logs are decoupled into two paths to eliminate write-lock contention under high-velocity ingestion:



Figure 2: Dual-data-layer write path.

- In-memory graph (PyG Data object or DGL DGLGraph) absorbs all high-frequency Spark writes with zero transaction overhead. This is the live operational graph.
- Neo4j receives periodic batch flushes (configurable; default 30s) and remains the persistent source of truth for historical querying and LangChain tool calls.
- TGN training reads directly from the in-memory object, bypassing Neo4j I/O during the hot training loop. This keeps the inference server well below 50 ms.

Implementation Note (Week 1). When designing the Neo4j schema, immediately include an `extraction_confidence` property on every edge (FLOAT, range [0, 1]). This must be present in the initial schema, or retrofitting it later will require a full graph migration.

3.1.1 Feasibility Evaluation

Feasibility	High
Novelty	Medium-High

Table 2: Feasibility evaluation, Step 1A.

The dual-data-layer approach was assessed as technically correct and well aligned with real-time cyber threat intelligence systems. Separating high-frequency streaming from persistent storage measurably reduces Neo4j write contention while enabling fast graph updates, and forms a strong foundation for all later phases. Most underlying components — Spark streaming, PyG/DGL, and Neo4j — are mature, well-documented technologies, and this combination is considered the single most reliable component of the plan. Dual-storage architectures are precedented in general streaming systems; their application to a Temporal Cyber Threat Knowledge Graph specifically is the engineering contribution retained here.

No structural changes to this step were found to be necessary. Where multi-process training is planned, the in-memory graph should be implemented using DGL's shared-memory features, which preserves the dual-layer design while improving scalability. Where laboratory GPU resources are constrained, a reduced configuration — a pure in-memory graph with periodic Neo4j snapshots used only for persistence — remains available as a scaled-down deployment of the same architecture.

3.2 1B — Uncertainty Quantification in Triplet Extraction

Rather than discarding low-confidence NLP extractions or passing them as if fully trusted, uncertainty is encoded explicitly as an edge weight attribute:

$$e(s, r, o, t) = \langle s, r, o, t \rangle, \quad c_{NLP} \in [0, 1]$$

where the quadruplet $\langle s, r, o, t \rangle$ carries an associated confidence score c_{NLP} , the softmax-normalised output of the SecureBERT relation classifier.

Confidence Range	Action	Rationale
$c \geq 0.90$	Full weight ($w = 1.0$)	High-confidence extraction; treated as ground truth
$0.70 \leq c < 0.90$	Partial weight ($w = c$)	Uncertain; down-weighted in TGN aggregation
$0.50 \leq c < 0.70$	Flagged ($w = 0.3$)	Borderline; human review queue + soft inclusion
$c < 0.50$	Rejected	Below threshold; logged but excluded from graph

Table 3: Confidence thresholding policy for extracted triplets.

Milestone 1: a populated Neo4j TKG with extraction confidence on all edges, an in-memory live graph object, and a UQ-aware extraction pipeline. The confidence scores themselves are a novel dataset contribution.

3.2.1 Feasibility Evaluation

Feasibility	High
Novelty	Medium

Table 4: Feasibility evaluation, Step 1B.

Encoding SecureBERT extraction confidence as a graph edge property is feasible, integrates naturally with downstream graph learning, and the thresholding policy in Table 3 is practical. SecureBERT (and comparable domain-specific encoders) is readily available, and confidence-aware graphs of this kind already have precedent in the uncertain knowledge graph literature — ConceptNet and NELL both attach confidence weights to triples, and embedding models such as UKGE handle confidence scores explicitly. Confidence is, however, currently used only as a static edge weight, which limits this step's novelty on a standalone basis.

3.2.2 Alternative Formulation — Confidence Decay and Threat Reliability Score

In place of a single static extraction_confidence value, the schema additionally carries a confidence_decay term, by which confidence diminishes as intelligence ages:

$$\text{confidence_decay} = \exp(-\lambda \cdot \Delta t)$$

Example — Edge: APT29 —uses→ PowerShell, extraction confidence = 0.91.

$$\text{Effective Confidence} = \text{Extraction Confidence} \times \text{Temporal Freshness}$$

in place of the original formulation Effective Confidence = Extraction Confidence. This allows the graph to automatically prioritise newer threat intelligence without deleting historical knowledge.

Feasibility	Very high — requires the addition of one further edge property
Novelty	High — dynamic temporal confidence propagation is materially more novel than static confidence weighting

Table 5: Feasibility evaluation, alternative formulation for Step 1B.

A further extension consolidates extraction confidence, source provenance, and temporal freshness into a single composite Threat Reliability Score:

$$\text{Threat Reliability} = \text{Extraction Confidence} + \text{Source Reputation} + \text{Temporal Freshness}$$

MITRE ATT&CK source + SecureBERT confidence + recent observation → Higher graph reliability → Better TGN embeddings → Better RL decisions

Figure 3: Propagation of the Threat Reliability Score through downstream phases.

This extension strengthens every downstream phase without changing the underlying dual-layer architecture, and is adopted as the reported contribution of Step 1B in this plan.

4 Phase 2 — TGN Forecasting (Weeks 5–8)

Confidence-weighted message aggregation; Zero-Day inductive test with structural fingerprint divergence evaluation.

4.1 2A — Confidence-Weighted Message Aggregation

$$h_v^{(\ell+1)}(t) = \sigma(W \cdot AGG\{c_{uv} \cdot m_{uv}(t) : u \in N(v), t_{uv} \leq t\})$$

where c_{uv} is the extraction confidence of edge (u, v) and $m_{uv}(t)$ is the time-encoded message. A low-confidence relation thus contributes proportionally less to the node's embedding, preventing unreliable NLP extractions from dominating the learned representation. The causally-constrained aggregation for target node v is:

$$z_v(t) = GNN(h_v(t^-), \{h_u(t^-) \mid u \in N(v), t_{uv} \leq t\})$$

4.1.1 Feasibility Evaluation

Feasibility	High
Novelty	Medium (incremental)

Table 6: Feasibility evaluation, Step 2A.

The modified aggregation function is a clean extension of standard TGN message passing and directly leverages the confidence scores established in Phase 1; implementation is straightforward in either PyTorch Geometric or DGL. Because it mostly propagates a signal already introduced in Step 1B, its novelty is incremental rather than independent — a relationship that is intentional and is documented as such: Phase 1 embeds uncertainty at data ingestion, Phase 2 propagates and refines it through forecasting, removing the redundancy concern raised in cross-phase review (Section 7) without requiring either step to be removed.

4.1.2 Alternative Formulation — Attention-Guided Confidence Weighting

In place of treating neighbour weight as confidence alone, an attention-guided variant is adopted as a drop-in replacement for simple multiplication:

$$\text{Neighbor Weight (current)} = \text{Confidence}$$

$$\text{Neighbor Weight (modified)} = \text{Confidence} \times \text{Attention Score} \times \text{Temporal Importance}$$

This allows the TGN to learn, simultaneously, which neighbour matters, how reliable it is, and how recent it is, with no architectural change required to the aggregation pipeline. Where training instability is observed under this formulation, gradient clipping and a learning-rate warm-up schedule are applied as standard mitigations.

4.2 2B — Zero-Day Inductive Test Protocol

- Label-novelty test: mask all “Ransomware” technique nodes; evaluate ranking at inference time. Target: Hits@10 $\geq$ 70%.
- Structure-novelty test: generate synthetic adversarial subgraphs with edge-type distributions not present in training data; evaluate whether TGN embeddings for synthetic nodes cluster near semantically similar known patterns. Metric: cosine similarity to nearest known-malicious cluster $>$ 0.65.

- Structural-divergence threshold: if cosine similarity to any known cluster falls below 0.40, the node is flagged as “structurally novel — requires human review” rather than producing a silent prediction. This converts a potential silent failure into a controlled, auditable escalation.

Implementation Note (Week 5). The FastAPI embedding server must accept batch matrix inputs (a list of node IDs and a shared timestamp), not just individual node lookups. A single matrix forward pass is orders of magnitude faster than sequential calls and is essential for keeping inference below 50 ms when the DDQN requests state updates during rollout.

4.2.1 Feasibility Evaluation

Feasibility	Medium-High
Novelty	Medium-High (standalone)

Table 7: Feasibility evaluation, Step 2B.

The two-tier (label-novelty and structure-novelty) test is judged innovative and addresses a real gap identified in Section 1.2 (Gap 1): standard inductive tests detect label novelty but not structural novelty. Synthetic adversarial subgraph generation is feasible but requires careful implementation, and generating realistic synthetic attack paths together with selecting a defensible structural-divergence threshold is expected to require iterative tuning rather than a single design pass. Used alone, however, synthetic graphs may not fully represent realistic cyber attacks, which is the principal limitation of the step as originally specified.

4.2.2 Alternative Formulation — Hybrid Zero-Day Evaluation

In place of synthetic adversarial subgraphs alone, three data sources are combined into a single graded evaluation:

- Synthetic graphs (as originally specified).
- Historical MITRE ATT&CK attack chains.
- CyberBattleSim attack trajectories.

Known attacks → Modified attacks → Generated attacks → Completely unseen attacks

Figure 4: Graded generalisation spectrum under the hybrid Zero-Day evaluation.

Feasibility	High — requires only additional evaluation datasets, no new modelling
Novelty	Very high — the hybrid evaluation is substantially stronger evidence for the Zero-Day claim than a synthetic-only test

Table 8: Feasibility evaluation, alternative formulation for Step 2B.

Where synthetic graph generation proves time-consuming within the available schedule, the protocol may instead begin from public datasets (DARPA OpTC, CIC-DDoS2019) with controlled perturbations applied, or fall back to simpler out-of-distribution detection — Mahalanobis distance in embedding space, graph autoencoder reconstruction error, or monitoring of prediction-confidence drift — ahead of building the full structural-divergence pipeline. A held-out set of known Zero-Day examples is reserved throughout to validate the chosen novelty threshold.

5 Phase 3 — Graph Reinforcement Learning (Weeks 9–13)

Action-masking layer; normalised reward function; adversarial self-play training environment.

5.1 3A — Topology-Constrained Action-Masking Layer

$$s_t = \{ z_t, A_{\text{valid}}(s_t) \}$$

$$\text{Masked}Q(s,a) = Q(s,a;\theta) \text{ if } a \in A_{\text{valid}}(s), \text{ else } -\infty$$

$A_{\text{valid}}(s)$ is computed by querying the Neo4j graph schema at each step: any action targeting a node tagged :CriticalInfrastructure, or with availability SLA above a configurable threshold, is masked out.

TGN State z_t → Topology Action Mask (Cypher: critical nodes → masked) → Double DQN $Q(s,a;\theta)$ → Action a^*_{valid}

Figure 5: Topology-constrained action-masking pipeline.

- Eliminates catastrophic action risk during early exploration entirely.
- Reduces effective action space size, dramatically accelerating convergence.
- The mask is dynamic — as network topology changes, so does the valid action set, teaching the agent to respect topology-aware constraints automatically.

Implementation Note (Week 9). Build CyberGraphEnv with action-masking from day one, using the Gymnasium `action_masks()` interface pattern. Retrofitting masking into a trained environment is extremely difficult and will cost more time than doing it correctly upfront.

5.1.1 Feasibility Evaluation

Feasibility	High
Novelty	High

Table 9: Feasibility evaluation, Step 3A.

Action masking via the Gymnasium interface is a standard and effective technique, and dynamic Cypher queries for critical-node identification are practical at runtime. Topology-constrained, dynamic graph-aware action masking is well suited to cyber defense specifically and is considered one of the two strongest novelty contributions of Phase 3. This step is retained without modification; the principal requirement is sequencing — the masking layer must be present from the first training run, consistent with the Week 9 implementation note above.

5.2 3B — Normalised Reward Function

$$R(s,a) = \alpha(\Delta AWF(a) / AWF_{\text{base}}) - \beta(OD_{\text{downtime}}) - \lambda \cdot 1_{FP}(a)$$

where $\alpha, \beta, \lambda \in [0,1]$ are tunable reward weights and $R(s,a) \in [-(\beta+\lambda), \alpha]$. The discount factor γ is reserved strictly for the RL temporal horizon.

Term	Raw Range	Normalisation	Bounded Range
ΔAWF	Integer (+1, +2, ...)	Divide by baseline AWF at episode start	[0, 1]
Downtime (OD)	% or continuous	Already percentage; used directly	[0, 1]
FP Indicator	Binary	No change needed	{0, 1}

Table 10: Reward normalisation mapping.

With all terms bounded in $[0, 1]$, the α , β , λ hyperparameters carry interpretable, comparable magnitudes, and the Pareto sweep over (α, β) produces a meaningful, unit-consistent frontier.

5.2.1 Feasibility Evaluation

Feasibility	High
Novelty	Medium (standard practice)

Table 11: Feasibility evaluation, Step 3B.

Normalisation to unitless ratios is theoretically sound and directly resolves the scale mismatch identified as Gap 2 (Section 1.2). The required mathematics are straightforward, and no implementation difficulty is anticipated. Reward normalisation of this kind is, however, standard reinforcement-learning practice, and the reward weights α , β , λ remain manually selected in the design as specified.

5.2.2 Alternative Formulation — Adaptive Reward Weighting

In place of fixed reward coefficients α , β , λ held constant throughout training, an adaptive weighting schedule is adopted, under which reward priorities evolve as training progresses:

Early training: larger penalty on false positives → Later training: larger weight on operational disruption

Figure 6: Adaptive reward-weight schedule across the training horizon.

Feasibility	High — requires reward scheduling only, no change to the reward formula itself
Novelty	Medium-High — more research-oriented than static reward tuning

Table 12: Feasibility evaluation, alternative formulation for Step 3B.

A small exploration bonus term based on graph entropy may additionally be added to encourage discovery of novel defense strategies. Where the resulting Pareto-front analysis becomes too complex to report within the available timeline, scalarised rewards with dynamic weight scheduling are retained as a lighter-weight substitute that preserves the same underlying design.

5.3 3C — Adversarial Self-Play Training Environment

Adversary Agent (Attacker RL) → Neo4j TKG Environment (Shared Graph State) → Defense Agent (Defender RL)

Figure 7: Self-play training environment with shared graph state.

- The Adversary Agent learns optimal lateral-movement paths through the Neo4j TKG, maximising asset-compromise score; its reward is the inverse of the Defense Agent's reward.
- Self-play continuously generates novel attack scenarios, replacing fixed DARPA OpTC replay with a dynamic, evolving threat model.
- Zero-Day stress testing becomes automatic: as the Adversary discovers new exploitation paths, it generates genuine in-distribution novel threats without hand-crafted synthetic test cases.
- Population-Based Training (PBT) maintains a league of past Adversary checkpoints, preventing the Defense Agent from over-fitting to the current Adversary policy.

Milestone 3: a self-play GRL cyber range producing dynamic, generative Zero-Day scenarios — a generative evaluation framework and independently publishable system contribution.

5.3.1 Feasibility Evaluation

Feasibility	Medium
Novelty	High — the strongest single novelty contribution in the plan

Table 13: Feasibility evaluation, Step 3C.

Graph-based self-play in a live cyber range has not been widely demonstrated, and the combination of topology-aware masking, a composite normalised reward, and attacker–defender self-play is genuinely innovative. Multi-agent reinforcement-learning frameworks exist for this purpose, and the CAGE challenges specifically explore multi-agent cyber defense, so the design is plausible in principle. The governing constraint is computational: training two RL agents simultaneously for a multi-week window, with Population-Based Training maintained throughout, represents the single most resource-intensive component of the plan, and carries the project's main convergence-instability risk (Risk 3, Section 1.1).

5.3.2 Alternative Formulation — Curriculum Self-Play and Graph Memory Replay

In place of training the Attacker RL and Defender RL agents simultaneously from initialisation, a staged curriculum is adopted, under which the defender is exposed to progressively more capable adversaries:

Static replay → Scripted attacker → Simple RL attacker → Full adversarial self-play

Figure 8: Curriculum self-play training stages.

Feasibility	Very high — materially easier to implement than simultaneous training from Week 9
Novelty	Very high — retains almost identical research value while substantially reducing implementation complexity

Table 14: Feasibility evaluation, alternative formulation for Step 3C.

A complementary mechanism, Graph Memory Replay, stores successful attack trajectories inside the graph itself, so that future RL episodes can replay the most successful attacks and the most failed defenses, enabling continual learning without any change to the underlying RL architecture. Where compute remains constrained even under the curriculum, self-play may additionally be run in a lower-fidelity graph environment with periodic validation against the full-fidelity Neo4j state, and distributed via Ray RLlib. Imitation learning from expert traces is reserved as a warm-start option ahead of the final full self-play stage.

6 Phase 4 — Interactive Copilot & X-AI (Weeks 14–16)

Rationale grounding via subgraph citation; counterfactual trajectory interrogation; confidence-gated human escalation.

6.1 4A — Rationale Grounding via Subgraph Citation

- Before invoking the LLM, the LangChain GraphQueryTool executes a Cypher query retrieving the subgraph nodes and edges that produced the maximum-influence message paths in the TGN forward pass.
- This subgraph is serialised as a Markdown table (Source Node, Relation, Target Node, Timestamp, Confidence) and injected directly into the prompt context.
- The system instruction constrains the LLM to justify every claim using only the nodes and edges in the table provided, introducing no threat-actor names, CVE identifiers, or techniques absent from the table.
- A post-generation citation-checking pass automatically flags any claim not traceable to a row in the table.

6.1.1 Feasibility Evaluation

Feasibility	High
Novelty	High

Table 15: Feasibility evaluation, Step 4A.

Subgraph extraction via Cypher and LangChain is reliable and directly reduces hallucination risk. Grounding LLM answers in a knowledge graph has been demonstrated in recent research, and systematically constraining the LLM to cited subgraph facts aligns with current best practice for hallucination reduction. This step is retained without modification; where stronger grounding is desired, influence-path highlighting — visualising the maximum-influence message paths directly — is added as a presentational enhancement rather than a design change.

6.2 4B — Interactive Policy Interrogation (Counterfactual Copilot)

Layer 4 is designed as a bidirectional security copilot rather than a static report generator.

Counterfactual Trajectory Interrogation. System: “Recommended action: VLAN Isolation of Node-47 [confidence: 0.61]. This is below the auto-execute threshold. Awaiting analyst review.” Analyst: “What would the attacker’s predicted path look like if I chose Rate-Limiting instead?” System: [triggers on-the-fly TGN link-prediction with Rate-Limiting applied] “If Rate-Limiting is applied, the highest-probability next-step attack path shifts to Node-12 (SQL Server) via T1078.003 (Valid Accounts) with probability 0.74, compared to 0.31 under VLAN Isolation. VLAN Isolation remains the recommended action.”

- CyberGraphEnv includes a hypothetical step(action) method that applies an action to a copy of the current in-memory graph state without committing to Neo4j.
- The TGN embedding server recomputes z_t^{hyp} on the modified graph copy and returns the new link-prediction distribution.
- LangChain formats the counterfactual distribution into a human-readable comparison table alongside the recommendation.

- All counterfactual queries and analyst decisions are logged to Neo4j as :AnalystIntervention nodes, creating a labelled dataset of human-override decisions for future supervised fine-tuning of the policy.

6.2.1 Feasibility Evaluation

Feasibility	Medium-High
Novelty	High

Table 16: Feasibility evaluation, Step 4B.

Counterfactual simulation using graph copies is technically feasible and highly innovative — allowing “what-if” policy questions on demand goes beyond static reporting and reflects best practice in human-in-the-loop AI design. The principal limitation is that recomputing the entire TGN forward pass after every analyst question adds latency and memory overhead that risks the system's own <50 ms / 2–3 second response targets.

6.2.2 Alternative Formulation — Localized Subgraph Simulation

In place of full graph recomputation on each analyst query, only the affected nodes and their immediate dependency chain are recomputed:

Only affected nodes → Neighbor embeddings → Attack path
--

Figure 9: Localized subgraph simulation scope.

Feasibility	Very high — a practical, narrowly-scoped recomputation
Novelty	Very high — maintains identical analyst-facing functionality at materially lower latency

Table 17: Feasibility evaluation, alternative formulation for Step 4B.

Where Localized Subgraph Simulation alone is insufficient, graph-diffing techniques are used to avoid full graph copying, and a caching layer is introduced for frequently queried counterfactuals where response time still exceeds the 2–3 second target. Interventions continue to be logged into a reusable Human Preference Dataset for future RLHF-style fine-tuning of the policy.

6.3 4C — Confidence-Gated Escalation Policy

Q-Value Confidence Gate	System Behaviour	LLM Role
$\max_a Q > \theta_high$	Auto-execute	Post-hoc report only
$\theta_c < \max_a Q \leq \theta_high$	Soft review (60s analyst window)	Full copilot mode
$\max_a Q \leq \theta_c$	Hard escalation — human decision required	Counterfactual mode

Table 18: Three-tier confidence-gated escalation policy.

6.3.1 Feasibility Evaluation

Feasibility	High
Novelty	Medium (as specified)

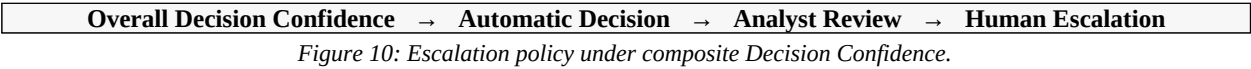
Table 19: Feasibility evaluation, Step 4C.

The three-tier policy is practical, interpretable, and well suited for SOC integration; no implementation difficulty is anticipated. As specified, escalation confidence depends only on the RL model's Q-values, which omits graph-side and historical reliability signals that the system otherwise produces.

6.3.2 Alternative Formulation — Composite Decision Confidence

In place of raw Q-values alone, the escalation gate is computed from a composite Decision Confidence term:

$$\text{Decision Confidence} = \text{RL Confidence} \times \text{Graph Confidence} \times \text{Prediction Stability} \times \text{Historical Decision Success}$$



Feasibility	High — a combination of outputs already produced by the system, requiring no new modelling
Novelty	Very high — substantially stronger than gating on Q-values alone

Table 20: Feasibility evaluation, alternative formulation for Step 4C.

Thresholds θ_{high} and θ_{c} are additionally made adaptive, based on recent system performance, via simple meta-learning rather than held fixed. Every analyst correction is fed back to update graph reliability, reward priorities, policy confidence, and copilot explanations — a Human Feedback Loop that allows the system to improve continuously while remaining entirely within the cyber-defense, graph-learning, and explainable-AI domain already targeted by the project.

7 Cross-Phase Findings

Read across all four phases, three structural patterns recur and are addressed here as a single cross-cutting analysis rather than four separate phase notes.

7.1 Redundant Steps

Step 1B (confidence-weighting at extraction) and Step 2A (confidence-weighting in TGN aggregation) are conceptually similar. This is treated as intentional rather than defective: Phase 1 embeds uncertainty at data ingestion, and Phase 2 propagates and refines that uncertainty through forecasting. The attention-guided weighting introduced in Section 4.1.2 is what differentiates the two stages in practice.

7.2 High Complexity Concentration

Self-play with Population-Based Training in Phase 3, and on-the-fly counterfactual recomputation in Phase 4, are the plan's two highest-complexity items, and both sit late in the schedule (Weeks 9–16), where there is least slack to recover from delays. Curriculum Self-Play (Section 5.3.2) and Localized Subgraph Simulation (Section 6.2.2) are adopted specifically to de-risk this concentration without reducing project scope.

7.3 Integration as the Project's Principal Novelty

No single phase in isolation is groundbreaking: dual-layer storage, confidence-weighted graphs, action-masked reinforcement learning, and grounded LLM copilots each have precedent individually. The genuine contribution of this project is the end-to-end integration of these components — NLP extraction, temporal knowledge graph, graph reinforcement learning, and explainable LLM copilot — with a single uncertainty signal and a self-play threat generator running through the entire pipeline. Reporting and evaluation are accordingly framed around this integration rather than around any individual component.

8 Consolidated Feasibility Findings

This section consolidates the per-step evaluations of Sections 3–6 into a single project-level assessment of feasibility, correctness, and novelty, together with the design refinements adopted in response.

8.1 Steps of Limited Standalone Novelty

Three steps were found to be correctly feasible and necessary, but only incrementally novel when considered in isolation: Step 1A and 1B (foundational ingestion and uncertainty encoding), Step 2A (confidence propagation, a direct consequence of 1B), and Step 3B prior to its adaptive-weighting extension (reward normalisation, standard RL practice). None of these steps were removed from the plan; each is retained as required scaffolding, with its associated refinement — Threat Reliability Score, attention-guided weighting, or adaptive reward scheduling — adopted as the reported contribution.

8.2 Steps Carrying Material Feasibility Risk

- Step 2B, in its synthetic-only form: Medium feasibility, owing to the realism limitations of purely synthetic adversarial subgraphs.
- Step 3C, in its full simultaneous self-play and PBT form: Medium feasibility, owing to the computational demand of training two reinforcement-learning agents concurrently within the available timeline.
- Step 4B, in its full-graph recomputation form: Medium-High feasibility, owing to the latency and memory overhead of recomputing the entire TGN forward pass per analyst query.

8.3 Adopted Alternative Formulations

Step	Limitation Identified	Alternative Formulation Adopted	Resulting Feasibility / Novelty
2B	Synthetic-only Zero-Day test	Hybrid evaluation: synthetic graphs combined with MITRE ATT&CK chains and CyberBattleSim trajectories	High / Very High
3C	Full self-play and PBT from Week 9	Curriculum Self-Play (static replay → scripted → simple RL → full self-play) with Graph Memory Replay	Very High / Very High
4B	Full TGN recomputation per query	Localized Subgraph Simulation, recomputing only affected nodes and neighbour embeddings	Very High / Very High

Table 21: Adopted alternative formulations and their resulting feasibility.

8.4 Secondary Fallback Positions

Each adopted alternative formulation carries its own documented fallback, so that no stage of the project is left without a working implementation path within the project timeline:

- Step 2B fallback: public datasets (DARPA OpTC, CIC-DDoS2019) with controlled perturbations, or simpler out-of-distribution detection (Mahalanobis distance, graph autoencoder reconstruction error, prediction-confidence drift).

- Step 3C fallback: a single scripted or heuristic adversary, or replayed attack logs, with Ray RLlib for distributed training where compute allows; full self-play is documented as a stretch objective if scheduling does not permit it.
- Step 4B fallback: a small set of pre-computed counterfactual scenarios served offline, with a caching layer for the most frequently asked analyst questions.

8.5 In-Process Modifications

Where a step was found to be sound but under-specified, the lower-risk position adopted is to modify the existing process rather than to substitute a different method:

- Step 1B: add `confidence_decay` and the Threat Reliability Score as schema-level edge properties — a single additional FLOAT field, with no change to the ingestion pipeline.
- Step 2A: replace the neighbour-weight term `Confidence` with `Confidence × Attention Score × Temporal Importance` — a formula change inside the existing aggregation function.
- Step 3B: replace fixed α, β, λ with a reward-weight schedule shifting emphasis from false positives early in training to operational disruption later.
- Step 4C: compute the escalation gate from the composite Decision Confidence term rather than raw Q-values, with analyst corrections fed back through the Human Feedback Loop.

8.6 Feasibility Status Summary

Step	Feasibility	Basis
1A	High	Mature components (Spark, PyG/DGL, Neo4j); mitigates a known production bottleneck.
1B	High	Confidence as edge weight is straightforward to add; decay/reliability extension is a one-field schema change.
2A	High	Clean extension of standard TGN aggregation; attention-guided variant is a drop-in formula change.
2B	Medium-High	Innovative two-tier test; synthetic-only data is the principal risk, resolved by hybrid evaluation.
3A	High	Standard, well-supported Gymnasium action-masking pattern; must be built first, not retrofitted.
3B	High	Straightforward normalisation mathematics; adaptive weighting is an enhancement, not a requirement.
3C	Medium	Most ambitious step; Curriculum Self-Play with Graph Memory Replay raises this to High with minimal value loss.
4A	High	Reliable Cypher and LangChain grounding pattern; directly reduces hallucination risk.
4B	Medium-High	Technically feasible; Localized Subgraph Simulation removes the principal latency/memory risk.
4C	High	Practical, interpretable three-tier policy; composite confidence strengthens it further.

Table 22: Feasibility status of all steps after the adopted refinements.

8.7 Domain-Fit Verification

Every refinement and alternative formulation adopted in Sections 3–6 — confidence decay, the Threat Reliability Score, attention-guided weighting, hybrid Zero-Day evaluation, Curriculum Self-Play, Graph Memory Replay, Localized Subgraph Simulation, adaptive reward weighting, the composite Decision Confidence term, and the Human Feedback Loop — was verified to operate strictly within the project's existing domain boundaries of temporal knowledge graphs, graph-based reinforcement learning, and explainable AI for cyber threat intelligence. None introduces a new modality, data type, or research area; each is a parameter, formula, scheduling, or evaluation-data change layered onto a component already specified in the plan. The project therefore remains feasible as a single, coherent, publishable system after every adopted modification.

9 Novelty Axes

This plan is structured around five clearly differentiated novelty contributions.

#	Novelty Axis	Status Following Feasibility Evaluation
1	Latent Vector State Space (z_t as RL state)	Unchanged — high feasibility, retained as specified.
2	Pareto Frontier Documentation (normalised AWF vs. OD)	Strengthened by adaptive reward weighting (Section 5.2.2).
3	Inductive Structural Novelty (two-tier Zero-Day test)	Strengthened by hybrid evaluation (Section 4.2.2).
4	Adversarial Self-Play Cyber Range	De-risked via Curriculum Self-Play and Graph Memory Replay (Section 5.3.2); novelty preserved.
5	Counterfactual Policy Interrogation	De-risked via Localized Subgraph Simulation (Section 6.2.2); novelty preserved.

Table 23: Novelty axes and their status following evaluation.

10 Simulation Environment Design

The training environment operates at two levels simultaneously.

10.1 Graph-Level Simulation (Primary)

CyberGraphEnv directly manipulates the in-memory PyG/DGL graph object. VLAN Isolation is modelled as an edge-deletion operation; Honey-Token Deployment adds a synthetic decoy node; Rate-Limiting reduces edge weights on outbound connection edges. The environment is intentionally graph-native rather than packet-level, matching the granularity at which the TGN operates.

10.2 Adversary Agent (Secondary, Self-Play)

The Adversary Agent provides dynamic event generation, eliminating dependence on static DARPA OpTC replays beyond the initial curriculum phase. CyberBattleSim (Microsoft Research) can serve as the underlying packet-level action executor where true network fidelity is required, with its graph state mapped bidirectionally to Neo4j node properties via a thin adapter layer.

Why not a full packet-level simulator? Full network simulators operate at a different granularity than the TGN and would require a complex, lossy mapping between packet-level states and graph embeddings. The graph-native environment preserves exact semantic alignment between the RL state (z_t) and action consequences, which is essential for the DDQN to learn meaningful policies. Packet-level fidelity is reserved for future work as a validation layer.

11 Conclusion

This action plan addresses key engineering risks and theoretical considerations across all four pipeline layers, and the feasibility evaluation conducted in Sections 3–8 confirms, phase by phase, that the architecture is implementable as designed. The five core design decisions — dual-layer data architecture, uncertainty-quantified triplet extraction, topology-constrained action masking, normalised reward formulation, and subgraph-grounded LLM rationale — together constitute a production-grade research system, and none require replacement.

Where individual steps carried feasibility or novelty risk — the synthetic-only Zero-Day test, full adversarial self-play with Population-Based Training, and full-graph counterfactual recomputation — this evaluation identified in-domain alternative formulations (hybrid Zero-Day evaluation, Curriculum Self-Play with Graph Memory Replay, and Localized Subgraph Simulation) that preserve the project's research value while materially reducing implementation risk. Combined with the lighter-weight refinements (confidence decay, attention-guided weighting, adaptive reward scheduling, composite Decision Confidence, and the Human Feedback Loop), the plan now carries a documented feasibility and novelty justification at every sub-step, not only at the phase level.

The result is a research trajectory with five clearly differentiated novelty axes, a rolling publication strategy covering three distinct venues, and an evaluation framework designed to withstand scrutiny at the level of IEEE S&P and ACM CCS.

Output	Target Venue	Core Contribution
Phase 1–2 pipeline	IEEE CNS / USENIX Security (workshop)	UQ-aware CTI knowledge graph with confidence-weighted TGN
Phase 3 (GRL + self-play)	ACM CCS / IEEE S&P	Self-play cyber range + DDQN on graph state + Pareto analysis
Phase 4 (copilot)	ACM CHI / SOUPS	Human-AI interaction study on counterfactual policy interrogation
Full system + artifacts	CCS / NDSS (Artifact)	Open-sourced end-to-end pipeline with Neo4j snapshot

Table 24: Rolling publication strategy.

Document — June 2026